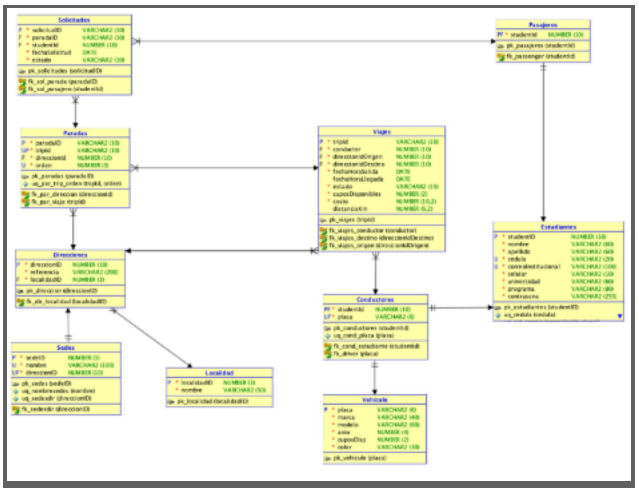
Documentación de Entrega 2 proyecto Bases de Datos



(10%) Análisis y revisión del modelo de datos.

Analicen la retroalimentación dada por parte del monitor en la Entrega 1 - diseño. A partir de esto:

a. Ajusten el modelo conceptual E/R e indiquen cuáles clases del modelo son actualizadas o creadas en esta entrega.

b. Actualicen el modelo relacional de la BD usando las reglas de transformación vistas en clase. Este debe quedar documentado en el formato Excel disponible para ello en BN.

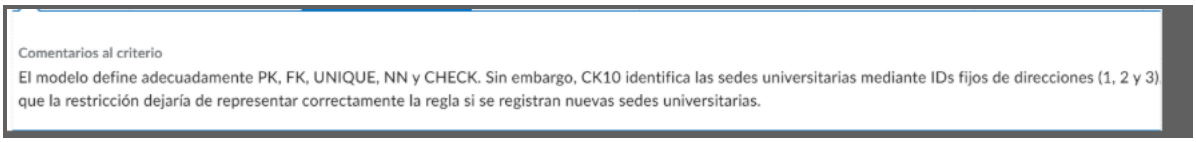
c. Verifiquen que cada relación del modelo esté en la Forma Normal de Boyce-Codd (BCNF).

De no estarlo, realicen los ajustes necesarios hasta cumplir con esta forma normal y documente dichos cambios. d. Anexen la documentación y modelos de este entregable

Tras hacer un análisis de nuestro desempeño en la entrega anterior, afortunadamente denotamos únicamente solo dos cambios a desarrollar, ambos en la sección final que corresponde a los Checks en las Etapa de prueba de la entrega 1 estos cambios se reducían a dos aspectos

Checkers	
num	uso
CK1	LENGHT 10
CK2	Debe terminar en '.edu.co'.
CK3	Formato AAA###: tres letras seguidas de tres dígitos.
CK4	Entre 1 y 12.
CK5	No puede ser menor a 1800

CK6	IN ('programado', 'en curso', 'completado', 'cancelado')
CK7	IN ('pendiente', 'aceptada', 'rechazada', 'cancelada')
CK8	Mayor o igual a 0.
CK9	Deben ser distintos. Un viaje no puede salir y llegar al mismo punto.
CK10	direccionIdOrigen IN (1,2,3) OR direccionIdDestino IN (1,2,3)
CK11	estado diferente de 'completado' OR (fechaHoraLlegada IS NOT NULL AND distanciaKm IS NOT NULL)
CK12	Mayor a 0.



Corrección del modelo: regla de sede universitaria, Check 10

Originalmente teníamos una restricción, Check 10 que identificaba las sedes universitarias mediante una serie de IDs fijos para así lograr cumplir la regla de negocio que garantizaba que “todo viaje sale o llega a una universidad”, en terminos de practicidad esta solución no consideraba la construcción de nuevas sedes universitarias creando un problema de actualización de datos en nuestra base de datos porque mezclábamos una regla de negocio con una restricción de dominio.

Para poder abarcar este problema decidimos crear una nueva Tabla/relación, denominada Sedes con (sedeID, nombre, direccionID como FK). Asi logramos contemplar que una dirección es sede universitaria si y solo si su direcciónID aparece en sedes.

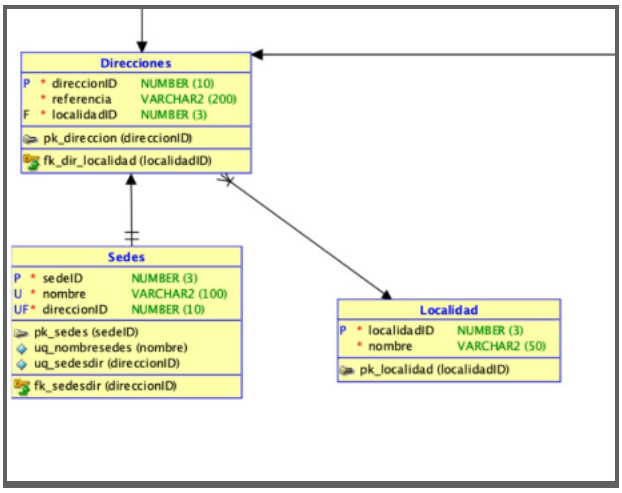


Figura 1, imagen de nueva entidad creada, Sedes con sus relaciones y atributos

Así no solo logramos mantener la estructura original de nuestra tabla de Viajes sino que también la información de que direcciones son sedes pasa a ser un dato almacenado y no una constante en una restricción como lo teníamos en nuestra entrega inicial. Asi podemos registrar nuevas sedes como un INSERT en la tabla de sedes, sin tener que modificar una restricción por cada sede nueva incluida en la base de datos.

Nuestra Racional y decisión de diseño

Usamos una FK Sedes dirección.ID y un unique para que podamos garantizar que cada dirección es como máximo una sede y viceversa cada sede es una dirección, la tabla ya se encuentra en BCNF con sus tres atributos (sedeID, nombre, direccionID como FK).

Consequentemente eliminamos Check 10

Usando el texto guía del curso “Database system concepts” de Abraham Silberschatz utilizamos un trigger, porque los Checks solo pueden evaluar columnas de la misma fila, sin tener acceso a otras tablas, es decir que para cumplir la regla de “origen o destino pertenece a sedes” implementamos un trigger que consulta al insertar o modificar un viaje

Asi también logramos mantener intacta nuestros atributos direccionidOrigen y direccionIdDestino que mantiene la estructura que nos ayuda mantener nuestro Check 9 (destino <> origen)

Corrección de RF5

“La cobertura de RF, RFC y RFA es amplia, pero RF5 modifica la regla del enunciado al permitir cancelar un viaje en curso sin exigir las dos horas de anticipación.”

La confusión yace en que el enunciado se contradice:

En el enunciado tenemos que RF5: “Un viaje puede cancelarse desde los estados programado o en curso, siempre con al menos dos horas de anticipación respecto a la hora de salida.” Sin embargo, como es posible cancelar un viaje con dos horas de anticipación si ya se encuentra en curso el viaje.

Triggers, que son, como funcionan y como los aplicamos

Para poder solucionar ambas problemáticas empleamos el concepto del texto guía Database systems concepts; Triggers. Por definición los triggers son un tipo de sentencia que permite al sistema ejecutar de manera automática como efecto secundario de la modificación de datos. Donde todo trigger tiene un evento catalizador, una condicional y una acción.

Para el Check 10: En el libro se muestra como hay un ejemplo en donde podemos usar un trigger con el propósito de comprobar que un valor exista en otra tabla, El CK10 nos obliga a que almenos uno entre el origen y el destino se encuentre en sedes, esto no se podía hacer con restricciones declarativas porque una llave foránea onliga que una columna exista en otra tabla y poner una FK en cada nos obligaría a que ambos (destino y origen) fueran sedes (caso que aunque puede psara no tiene sentido) y los checks que fueron usados anteriormente solo funcionan si por decisión de diseño no hay escalabilidad de la aplicación (ósea no podemos ingresar mas sedes con el paso del tiempo)

```
create trigger timeslot_check1 after insert on section
referencing new row as nrow
for each row
when (nrow.time_slot_id not in (
        select time_slot_id
        from time_slot)) /* time_slot_id not present in time_slot */
begin
  rollback
end;

create trigger timeslot_check2 after delete on timeslot
referencing old row as orow
for each row
when (orow.time_slot_id not in (
        select time_slot_id
        from time_slot) /* last tuple for time_slot_id deleted from time_slot */
  and orow.time_slot_id in (
        select time_slot_id
        from section)) /* and time_slot_id still referenced from section*/
begin
  rollback
end;
```

Figure 5.9 Using triggers to maintain referential integrity.

Ejemplo del libro para mantener integridad referencial, pagina 207, capitulo 5.3

El trigger trg_viajes_toca_sede usa la sentencia BEFORE INSERT OR UPDATE OF direccionIdOrigen, direccionIdDestino ON Viajes, para que el viaje invalido no se pueda almacenar cuando se intente añadir a Viajes.

```
SELECT COUNT(*)
  INTO v_cantidad_sedes
  FROM Sedes
 WHERE direccionID IN (:NEW.direccionIdOrigen, :NEW.direccionIdDestino);
```

Figura 2, Querie dentro del trigger que cuenta cuantas de los ids de sedes se encuentran en una tupla de viajes

Esta sentencia principalmente dentro del trigger sirve el rol de recorrer la tabla Sedes y se queda con las filas cuya direcciónId sea igual al origen o al destino de una tupla determinada en viajes y hace cuenta de cuantas apariciones hay {0,1,2} , este numero se guarda en la variable local de

v_cantidad sedes, donde a partir de una condicional determinamos si es valido o no el viaje, en caso que el contador sea cero tiramos un error y un mensaje que indica que la tupla de viaje no es valida para ingresar.

Es decir la lógica es: crear una variable contador que incrementa por cada sede encontrada en una tupla de viajes, si el contador es 0, la tupla no es valida para poder ingresar/estar en la tabla de viajes.

Para el RF5: Teníamos que garantizar la regla nuestra aplicación debía asegurarse que:

- Regla 1 La sequencia de los estados de viaje fueran programado, en curso & finalmente completado
- Regla 2 Los viajes completados o cancelados son inmutables
- Regla 3 Un viaje se puede cancelar si y solo si encuentra en estado de 'programado' o 'en curso' siempre con almenos dos horas de antelación a la hora de salida
- Regla 4 Cuando un viaje se cancela las reservas consigo quedan canceladas también

No podíamos usar restricciones declarativas porque:
Para la secuencia de los estados de cada viaje, es decir el saber el estado viejo y nuevo de una tupla en particular
Para poder usar la hora actual necesitamos usar SYSDATE, dato que no soporta CHECK
La cancelación de las reservas también modifica la tabla de solicitudes, por lo que el CHECK solo nos ayudaría con validación, no actualización

¿Como funciona el código?

Podemos dividir el código en 3 bloques:

El primer bloque se asegura de los estados finales, si un estado en completado o cancelado esa tupla no se puede mutar mas,

El segundo bloque es cuando el estado anterior fue programado o en curso donde usamos la funciona sysdate para estar la hora en la que se ejecuta la reserva de cupo , si yace dentro del

margen de tiempo cambia el estado de la solicitud de un usuario si esta esta en un estado ‘ pendiente’ o ‘aceptada’
El Tercer bloque garantiza que la secuencia valida se mantenga, Programado- en curso, en curso- completado

2. (25%) Implementación del esquema de la base de datos

La única cosa notable de la implementación de la bases de datos fue que se nos ilvidop incluir los credenciales de los estudiantes para que tengan acceso a la aplicación sea como docuctores o pasajeros por lo que utilizamos el siguiente código para alterar la tabla ESTUDIANTES:

```
ALTER TABLE Estudiantes ADD (passwordHash VARCHAR2(64));

UPDATE Estudiantes
   SET passwordHash = RAWTOHEX(STANDARD_HASH('estudiante123', 'SHA256'));

UPDATE Estudiantes
   SET passwordHash = RAWTOHEX(STANDARD_HASH('conductor123', 'SHA256'))
 WHERE studentId IN (SELECT studentId FROM Conductores);

COMMIT;}
```

&

```
ALTER TABLE Estudiantes ADD (contrasena VARCHAR2(30));

UPDATE Estudiantes SET contrasena = 'estudiante123';

UPDATE Estudiantes
   SET contrasena = 'conductor123'
 WHERE studentId IN (SELECT studentId FROM Conductores);

COMMIT;
```

Documentación de las consultas

Las consultas de los requerimientos RFC1 a RFC3 y RFA1 a RFA2 están en el archivo sql/4_consultas.sql. En la aplicación, cada una se ejecuta como fuente SQL de una región de APEX, con los valores de los filtros tomados de los campos de la página.

Criterios comunes

Filtro de fechas. Todas las consultas con periodo usan el rango [inicio, fin + 1 día):

```
fecha >= TRUNC(TO_DATE(:fecha_ini, 'YYYY-MM-DD'))
AND fecha < TRUNC(TO_DATE(:fecha_fin, 'YYYY-MM-DD')) + 1
```

La función TRUNC se aplica al parámetro y no a la columna. Así el filtro incluye el último día completo y no se pierde el uso de índices sobre la columna de fecha.

El periodo no se mide con la misma fecha en todas las consultas. En RFC1 y RFC2 se filtra por la fecha en que se hizo la solicitud (fechaSolicitud); en RFC3, por la fecha de salida del viaje (fechaHoraSalida).

Localidades sin datos. En RFC1 y RFC2 la consulta parte de Localidad con LEFT JOIN, para que las localidades sin solicitudes aparezcan con valor 0 en lugar de desaparecer.

Usuario conectado. Las consultas de los tableros (RFA1 y RFA2) filtran con la variable de aplicación G_STUDENT_ID, que se llena después de iniciar sesión. En 4_consultas.sql aparece un identificador fijo (conductor 1, pasajero 100) para poder probarlas en SQL Commands.

RFC1 - Demanda de solicitudes por localidad en un periodo

Cuántas solicitudes de cupo se hicieron en cada localidad de Bogotá dentro de un rango de fechas.

Tablas. Localidad, Direcciones, Paradas, Solicitudes.

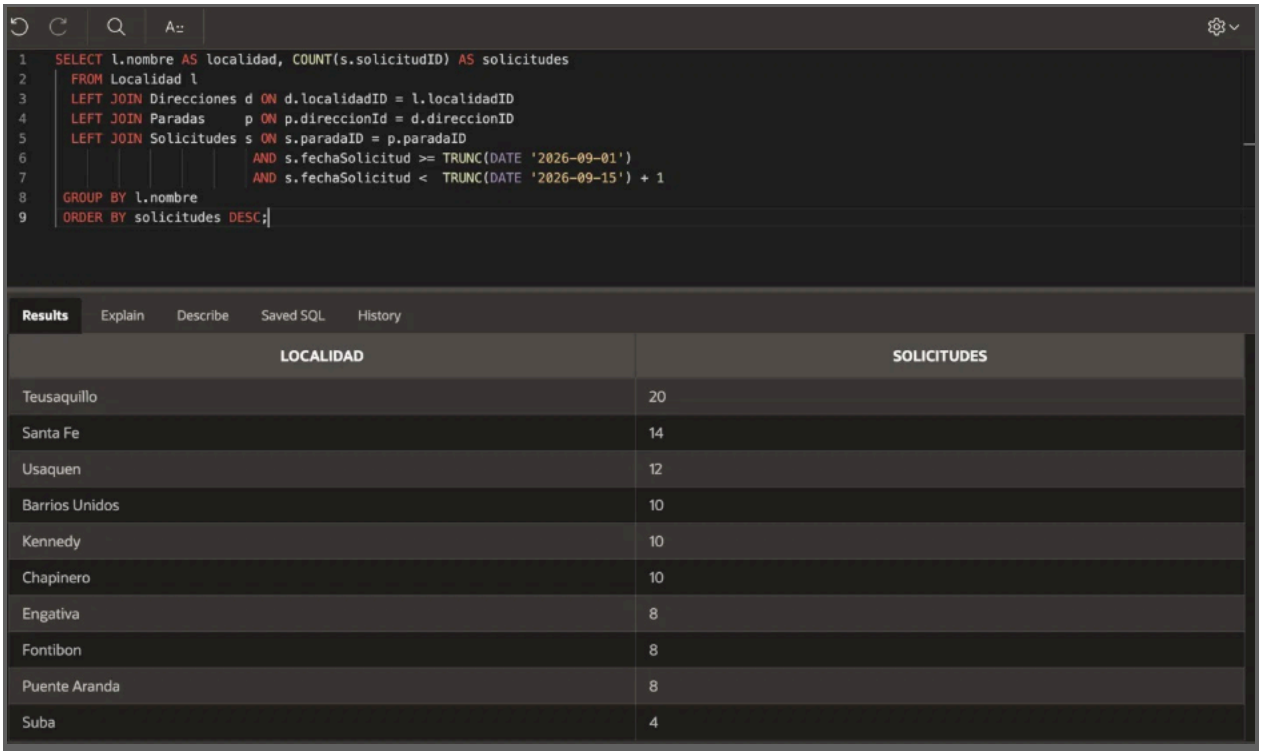
Atributos de los joins.

- Localidad.localidadID = Direcciones.localidadID
- Direcciones.direccionID = Paradas.direccionId
- Paradas.paradaID = Solicitudes.paradaID

Tipo de join y justificación. LEFT JOIN desde Localidad en los tres enlaces. Así las localidades sin solicitudes salen con 0. El filtro de fechas va en la cláusula ON de Solicitudes y no en el WHERE: en el WHERE el join se volvería interno y esas localidades desaparecerían.

Parámetros en la aplicación. :P5_FECHA_INI y :P5_FECHA_FIN. Se presenta como gráfica de barras (página 5).

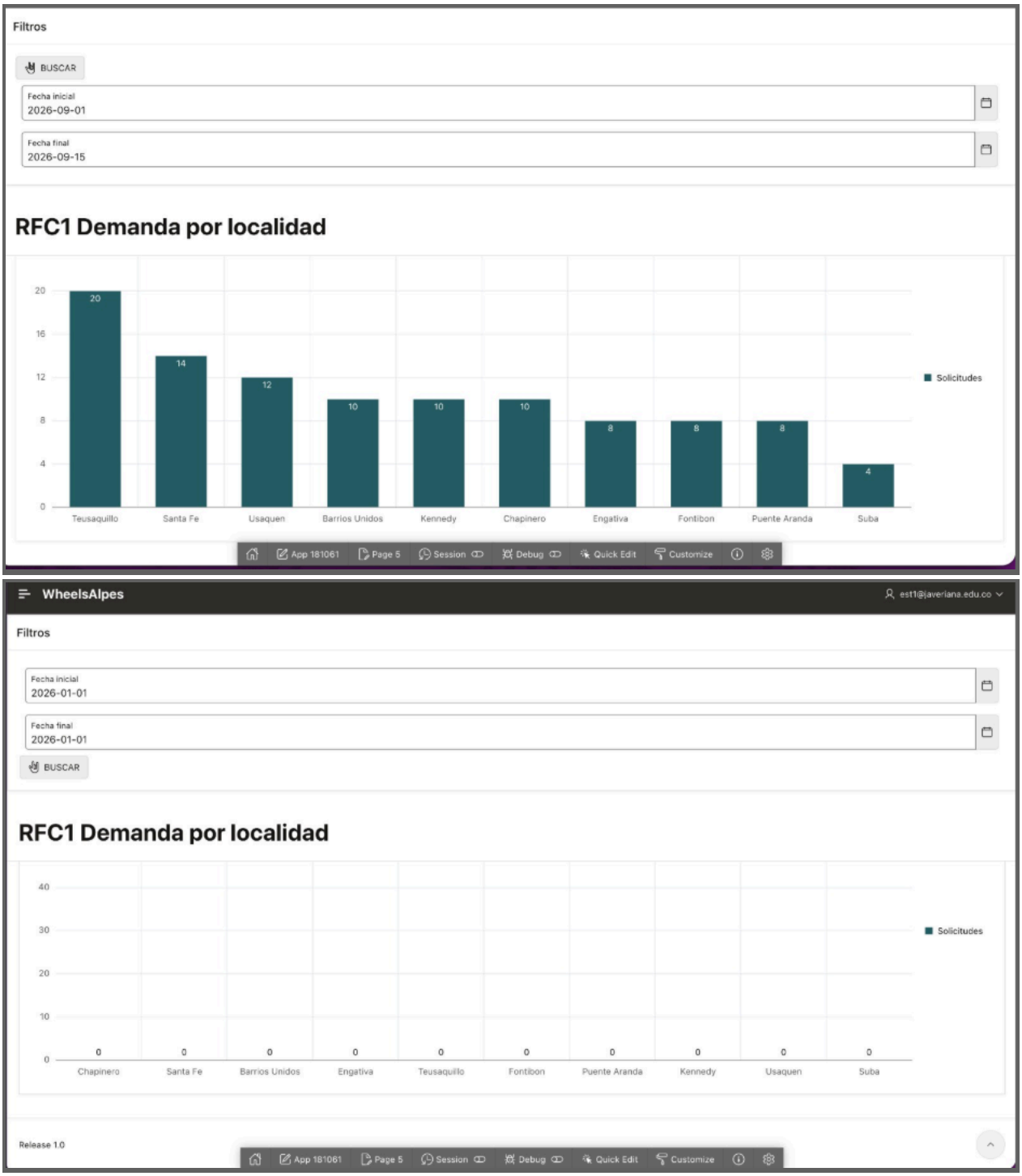
Validación. Con el rango 2026-09-01 a 2026-09-15, la consulta en SQL Commands devuelve las 10 localidades, con un total de 104 solicitudes. La gráfica de la aplicación muestra los mismos valores. Con un rango sin datos (2026-01-01 a 2026-01-01) las 10 localidades salen en 0.



```
SELECT l.nombre AS localidad, COUNT(s.solicitudID) AS solicitudes
  FROM Localidad l
  LEFT JOIN Direcciones d ON d.localidadID = l.localidadID
  LEFT JOIN Paradas     p ON p.direccionId = d.direccionID
  LEFT JOIN Solicitudes s ON s.paradaID = p.paradaID
                         AND s.fechaSolicitud >= TRUNC(DATE '2026-09-01')
                         AND s.fechaSolicitud <  TRUNC(DATE '2026-09-15') + 1
GROUP BY l.nombre
ORDER BY solicitudes DESC;
```

Results | Explain | Describe | Saved SQL | History

LOCALIDAD	SOLICITUDES
Teusaquillo	20
Santa Fe	14
Usaquen	12
Barrios Unidos	10
Kennedy	10
Chapinero	10
Engativa	8
Fontibon	8
Puente Aranda	8
Suba	4



RFC2 - Ingresos estimados por localidad y periodo

Qué responde. Cuántos ingresos generan las solicitudes aceptadas en cada localidad, en un rango de fechas. El ingreso de una solicitud es el costo del viaje al que pertenece.

Tablas. Localidad, Direcciones, Paradas, Viajes, Solicitudes.

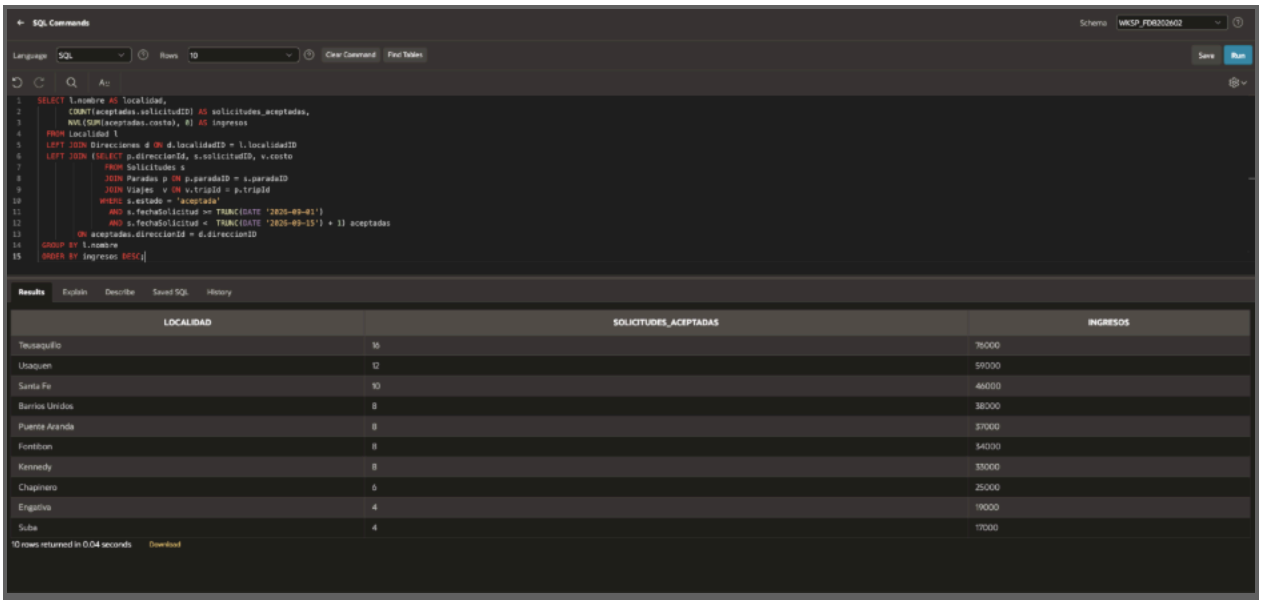
Atributos de los joins.

- Localidad.localidadID = Direcciones.localidadID
- Solicitudes.paradaID = Paradas.paradaID
- Paradas.tripId = Viajes.tripId
- Paradas.direccionId = Direcciones.direccionID

Tipo de join y justificación. Una subconsulta de solicitudes aceptadas une Solicitudes, Paradas y Viajes con INNER JOIN, porque solo interesan las solicitudes que existen y tienen viaje. Esa subconsulta se une a Localidad y Direcciones con LEFT JOIN, para que las localidades sin ingresos aparezcan con 0 gracias a NVL(SUM(...), 0).

Parámetros en la aplicación. :P6_FECHA_INI y :P6_FECHA_FIN. Gráfica de barras (página 6).

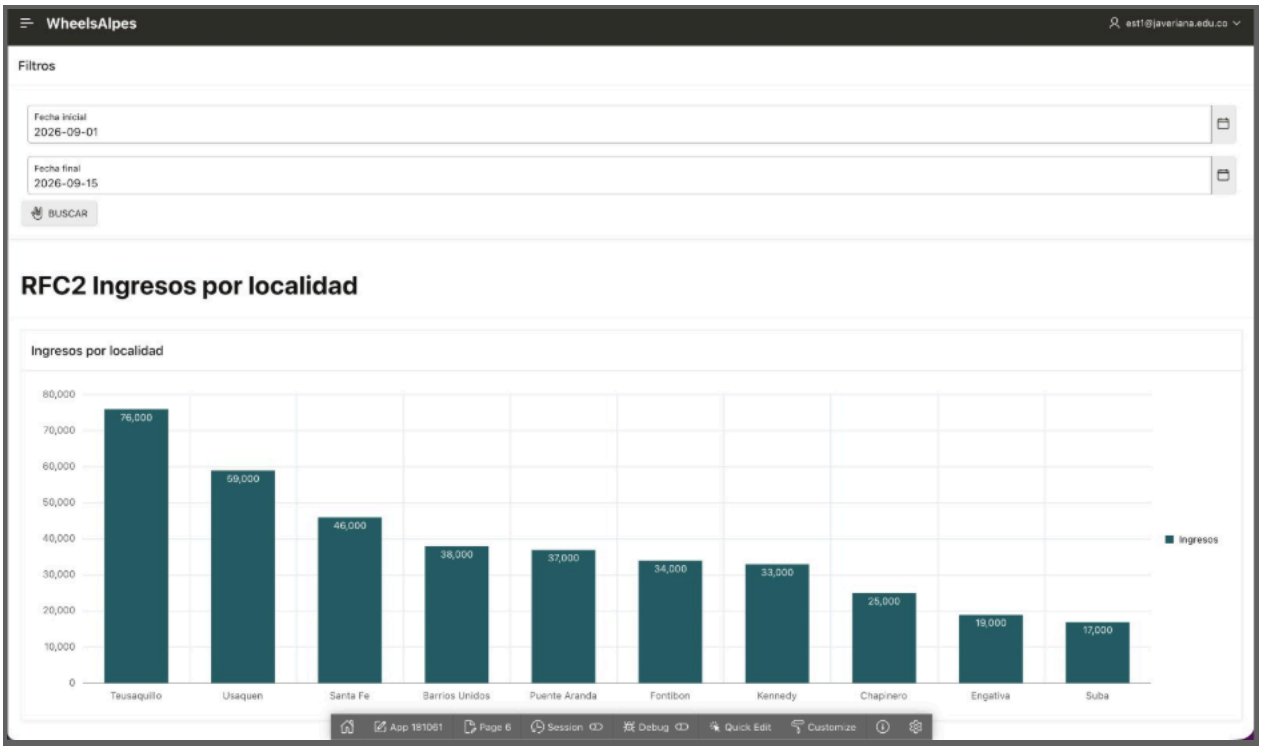
Validación. Con el rango del 1 al 15 de septiembre, la consulta y la aplicación muestran los mismos ingresos por localidad (Teusaquillo con el mayor valor, seguido de Usaquén).



```
SELECT l.nombre AS localidad,
       COUNT(aceptadas.solicitudID) AS solicitudes_aceptadas,
       NVL(SUM(aceptadas.costo), 0) AS ingresos
  FROM Localidad l
  LEFT JOIN Direcciones d ON d.localidadID = l.localidadID
  LEFT JOIN (SELECT p.direccionId, s.solicitudID, v.costo
               FROM Solicitudes s
               JOIN Paradas p ON p.paradaID = s.paradaID
               JOIN Viajes  v ON v.tripId = p.tripId
              WHERE s.estado = 'aceptada'
                AND s.fechaSolicitud >= TRUNC(DATE '2026-09-01')
                AND s.fechaSolicitud <  TRUNC(DATE '2026-09-15') + 1) aceptadas
         ON aceptadas.direccionId = d.direccionID
 GROUP BY l.nombre
 ORDER BY ingresos DESC;
```

LOCALIDAD	SOLICITUDES_ACEPTADAS	INGRESOS
Teusaquillo	16	76000
Usaquen	12	59000
Santa Fe	10	46000
Barrios Unidos	8	38000
Puente Aranda	8	37000
Fontibon	8	34000
Kennedy	8	33000
Chapinero	6	25000
Engativa	4	19000
Suba	4	17000

10 rows returned in 0.04 seconds



RFC3 - Viajes por conductor a través del tiempo

Cuántos viajes hizo cada conductor por día o por mes, con filtros opcionales por conductor y por localidad.

Tablas. Viajes y Estudiantes. Paradas y Direcciones solo en el filtro opcional de localidad.

Atributos de los joins.

- Viajes.conductor = Estudiantes.studentId
- Filtro de localidad: Paradas.direccionId = Direcciones.direccionID

Tipo de join y justificación. INNER JOIN con Estudiantes, porque todo viaje tiene un conductor. El filtro de localidad usa IN con una subconsulta y no un JOIN: un viaje con varias paradas en la misma localidad se contaría varias veces si se uniera directamente. No se incluyen los viajes cancelados (estado <> 'cancelado').

Parámetros en la aplicación (página 7). Los filtros están en la región “Filtros” y la consulta de la gráfica los lee con variables de página:

- :P7_FECHA_INI y :P7_FECHA_FIN: rango de fechas (campos de fecha, formato YYYY-MM-DD).
- :P7_CONDUCTOR y :P7_LOCALIDAD: listas desplegables con la opción “- Todos -”. Cuando se elige “Todos”, el valor queda vacío y la condición (:P7_CONDUCTOR IS NULL OR v.conductor = :P7_CONDUCTOR) deja pasar todos los registros. La de localidad funciona igual.
- :P7_FORMATO: lista “Ver por”, con YYYY-MM-DD para el periodo diario y YYYY-MM para el mensual. Se usa en TO_CHAR(fechaHoraSalida, :P7_FORMATO).

El botón BUSCAR envía los cinco campos y recarga la gráfica.

Presentación. Gráfica de barras apiladas: el eje horizontal es el periodo y cada conductor es una serie con su color, por lo que la altura de cada barra es el total de viajes de ese día o mes.

Validación.

- **Con filtros.** Con el conductor 1 y la localidad 2 en octubre de 2026, la consulta en SQL Commands devuelve 2 filas (2026-10-21 y 2026-10-22), igual que la aplicación.
- **Sin filtros.** Con el rango del 2026-10-20 al 2026-10-26, conductor y localidad en “Todos” y agrupado por día, la gráfica muestra 2, 2, 3, 1, 5, 3 y 4 viajes, con un total de 20. La consulta en SQL Commands devuelve los mismos valores por día.

SQL Commands

Schema: WKSP_FD8202802

Language: SQL | Rows: 10 | Clear Command | Find Tables | Save | Run

```
SELECT TO_CHAR(fechaHoraSalida, 'YYYY-MM-DD') AS periodo,
       COUNT(*) AS viajes
  FROM Viajes
 WHERE estado <> 'cancelado'
   AND fechaHoraSalida >= TRUNC(DATE '2026-10-20')
   AND fechaHoraSalida <  TRUNC(DATE '2026-10-26') + 1
 GROUP BY TO_CHAR(fechaHoraSalida, 'YYYY-MM-DD')
 ORDER BY periodo;
```

Results | Explain | Describe | Saved SQL | History

PERIODO	VIAJES
2026-10-20	2
2026-10-21	2
2026-10-22	3
2026-10-23	1
2026-10-24	5
2026-10-25	3
2026-10-26	4

rows returned in 0.00 seconds | Download

Filtros

Fecha inicial: 2026-10-20

Fecha final: 2026-10-26

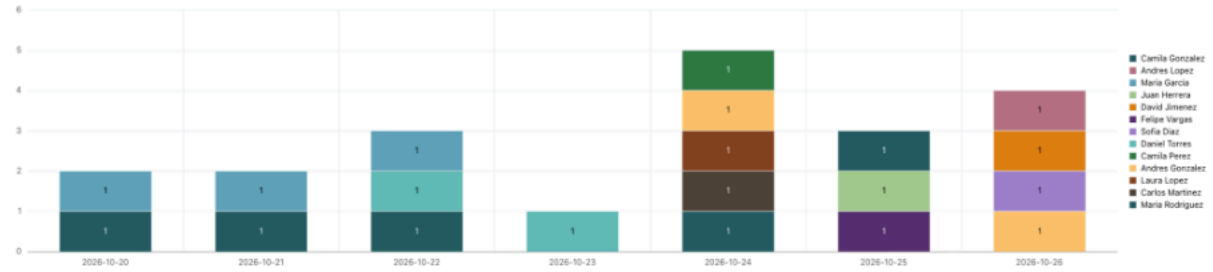
Conductor: - Todos -

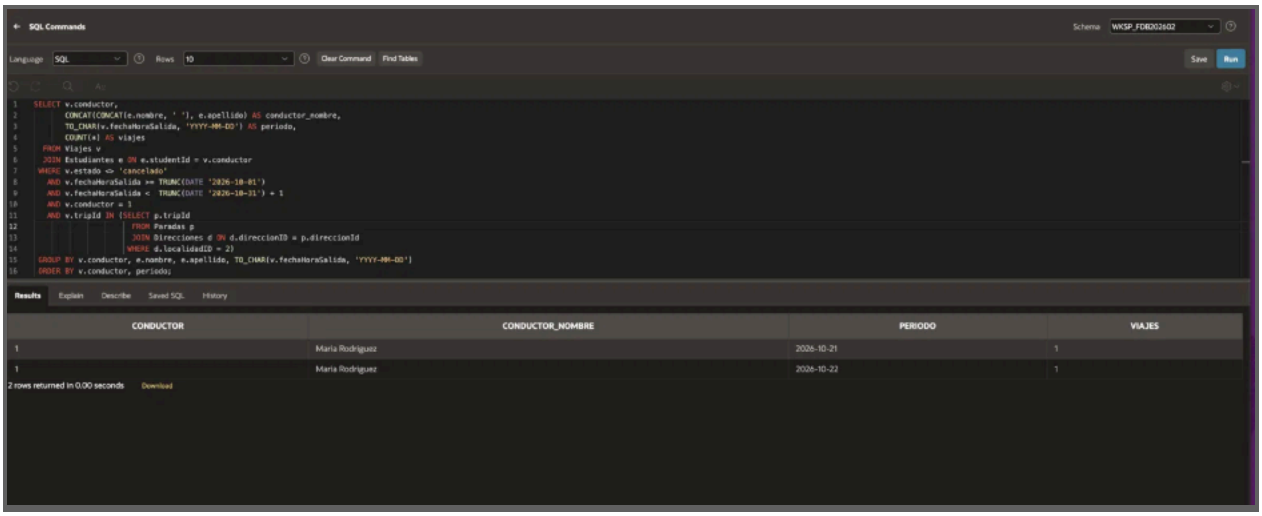
Localidad: - Todos -

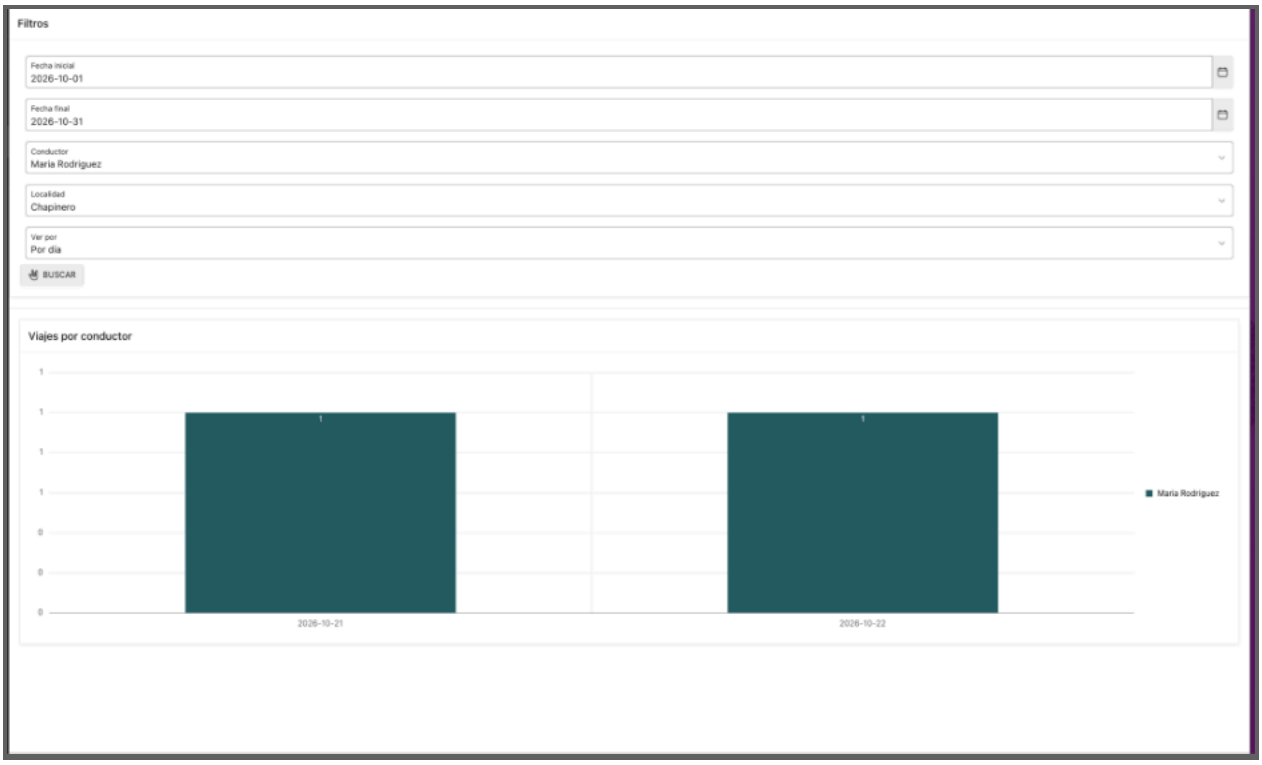
Ver por: Por día

BUSCAR

Viajes por conductor







RFA1 - Tablero de control del conductor

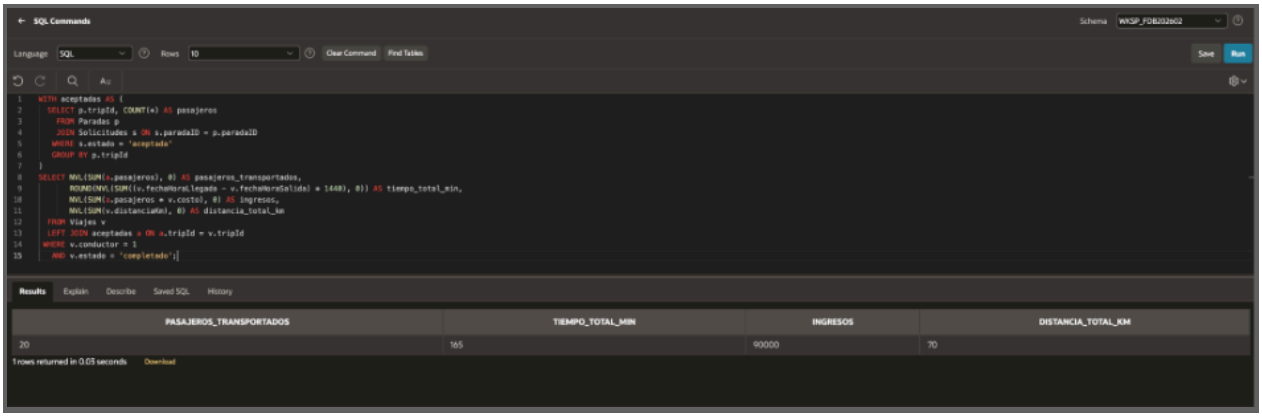
Cuatro indicadores del conductor conectado, calculados sobre sus viajes completados: pasajeros transportados, tiempo total en minutos, ingresos y distancia total.

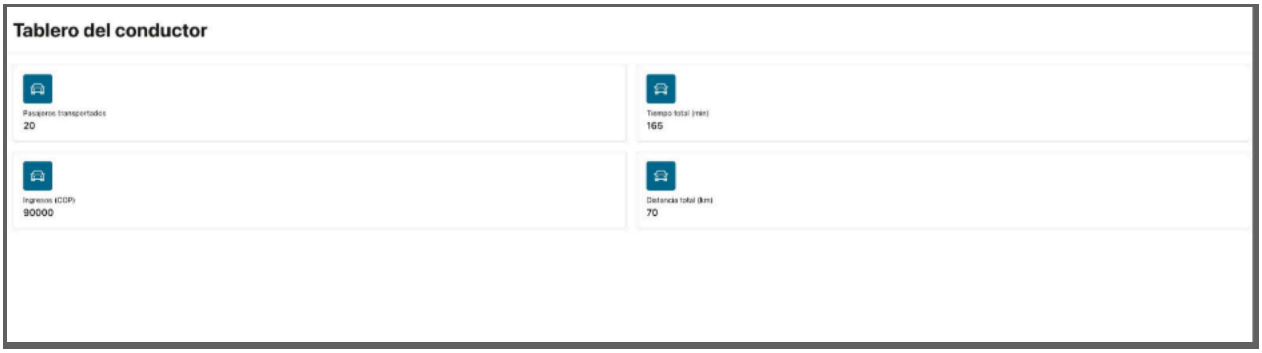
- Tablas. Viajes, Paradas, Solicitudes.
- Atributos de los joins.
- Solicitudes.paradaID = Paradas.paradaID
- Paradas.tripId = Viajes.tripId

Tipo de join y justificación. Los pasajeros aceptados se cuentan por viaje en una subconsulta que une Paradas con Solicitudes mediante INNER JOIN, porque solo interesan las solicitudes aceptadas que existen. Esa subconsulta se une a Viajes con LEFT JOIN: un viaje completado sin pasajeros no debe desaparecer. Contar antes de unir evita que el tiempo y la distancia de un viaje se sumen una vez por cada pasajero. Los ingresos son pasajeros × costo, y el tiempo es (llegada − salida) × 1440 minutos. En 4_consultas.sql la consulta además une Estudiantes con INNER JOIN (Viajes.conductor = Estudiantes.studentId), solo para mostrar el nombre del conductor; la región de la aplicación filtra directamente por G_STUDENT_ID.

Parámetro en la aplicación. :G_STUDENT_ID (página 2). Se muestra como cuatro tarjetas.

Validación. Para el conductor 1, la consulta y la aplicación muestran 20 pasajeros transportados, 165 minutos, 90.000 en ingresos y 70 km.





RFA2 - Tablero de control del pasajero

Qué responde. Cinco indicadores del pasajero conectado, sobre sus solicitudes aceptadas en viajes completados: viajes tomados, gastos totales, distancia promedio, tiempo promedio y zona más frecuente.

Tablas. Solicitudes, Paradas, Viajes. Direcciones y Localidad para la zona más frecuente.

Atributos de los joins.

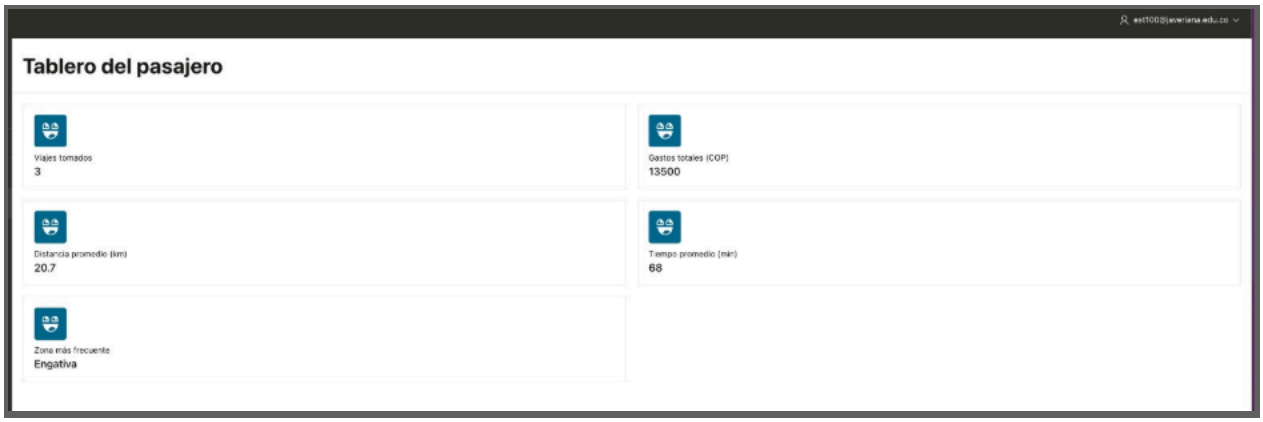
- Solicitudes.paradaID = Paradas.paradaID

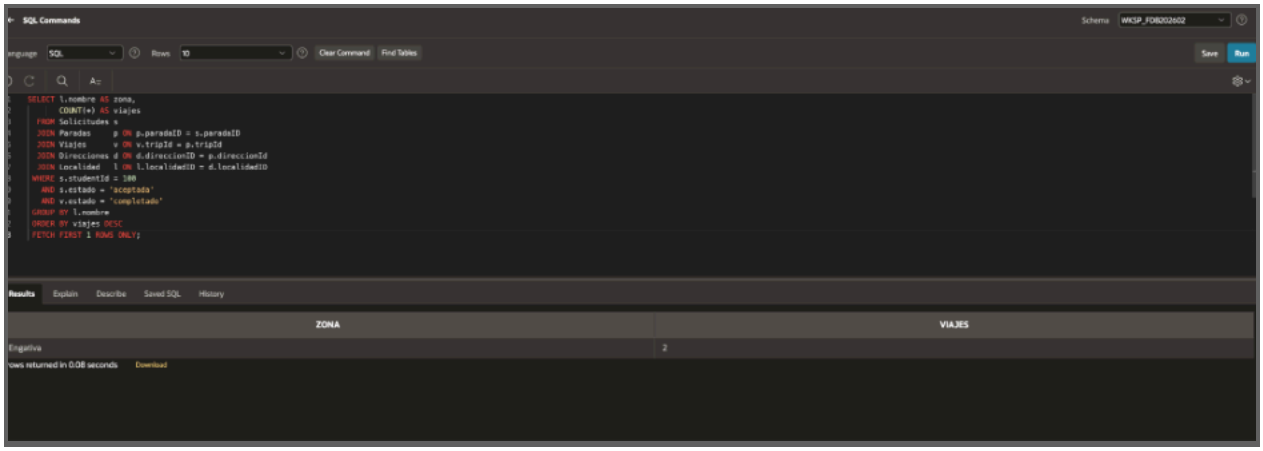
- Paradas.tripId = Viajes.tripId
- Paradas.direccionId = Direcciones.direccionID
- Direcciones.localidadID = Localidad.localidadID

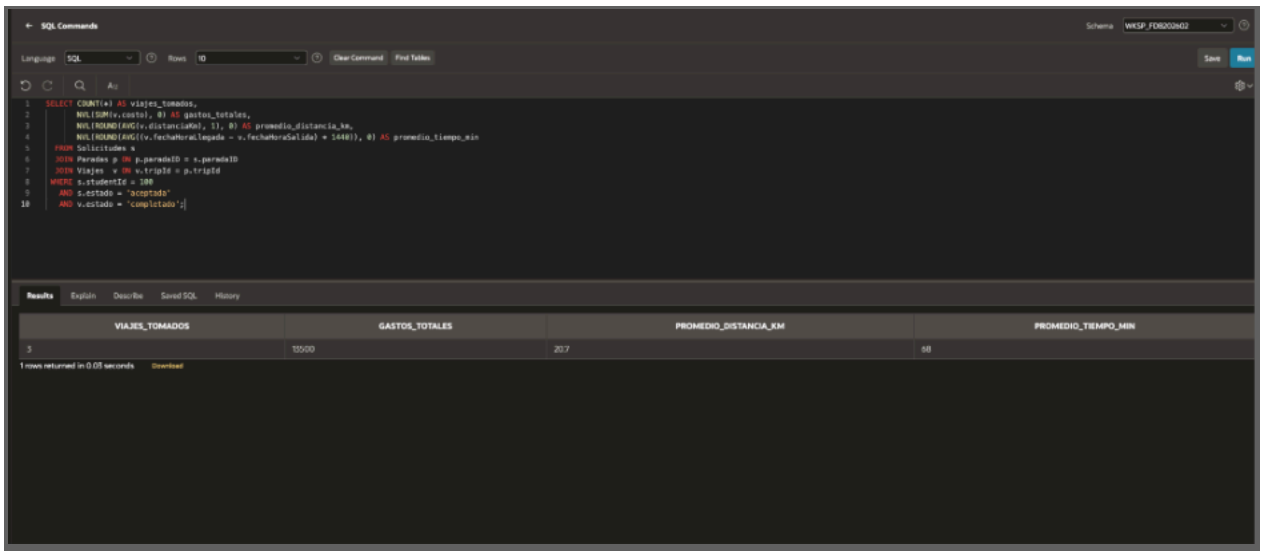
Tipo de join y justificación. INNER JOIN en todos los enlaces, porque solo cuentan las solicitudes aceptadas que llegan a un viaje completado. La zona más frecuente se obtiene agrupando por localidad, ordenando de forma descendente por cantidad y tomando la primera fila (FETCH FIRST 1 ROWS ONLY). Si dos localidades empatan, se muestra una sola.

Parámetro en la aplicación. :G_STUDENT_ID (página 8). Se muestra como cinco tarjetas.

Validación. Para el pasajero 100, la consulta y la aplicación muestran 3 viajes, 13.500 de gastos, 20,7 km y 68 minutos de promedio, y Engativá como zona más frecuente.

- 





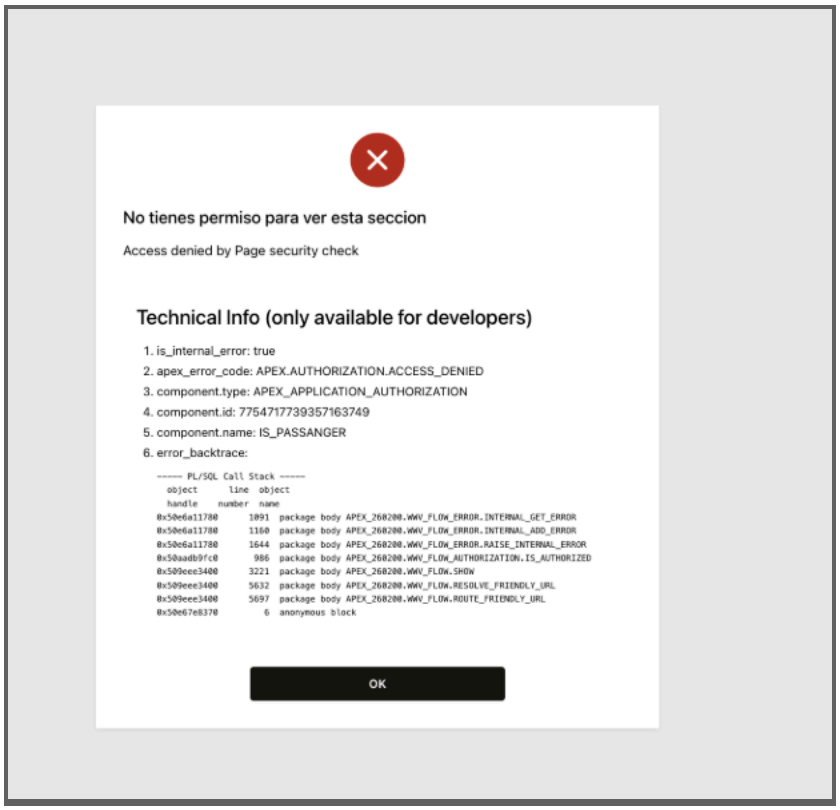
Documentación Desarrollo Aplicación

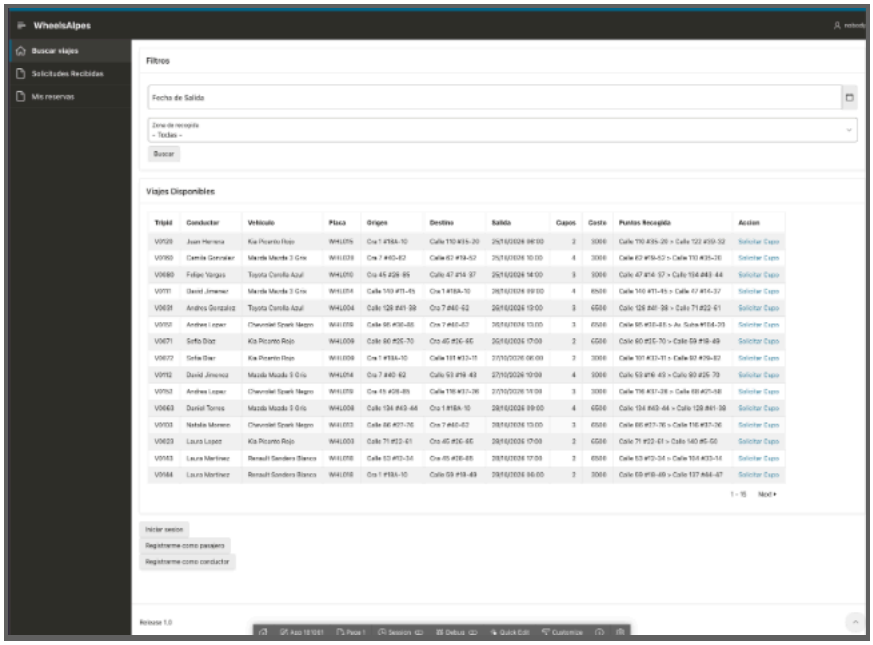
- Para la creación de la aplicación utilizamos varios esquemas de autenticación como parte de la integridad y privacidad de los datos de nuestros usuarios. Como comentamos antes, pasamos a tener una columna contrasena en estudiantes y un script sql que los autentique según esa columna en el login. Además de ese esquema

incorporamos is_driver e is_passanger para que solo aquellos con, por así decirlo, las ‘credenciales’ puedan acceder a las páginas que les habilitemos.

En concreto:

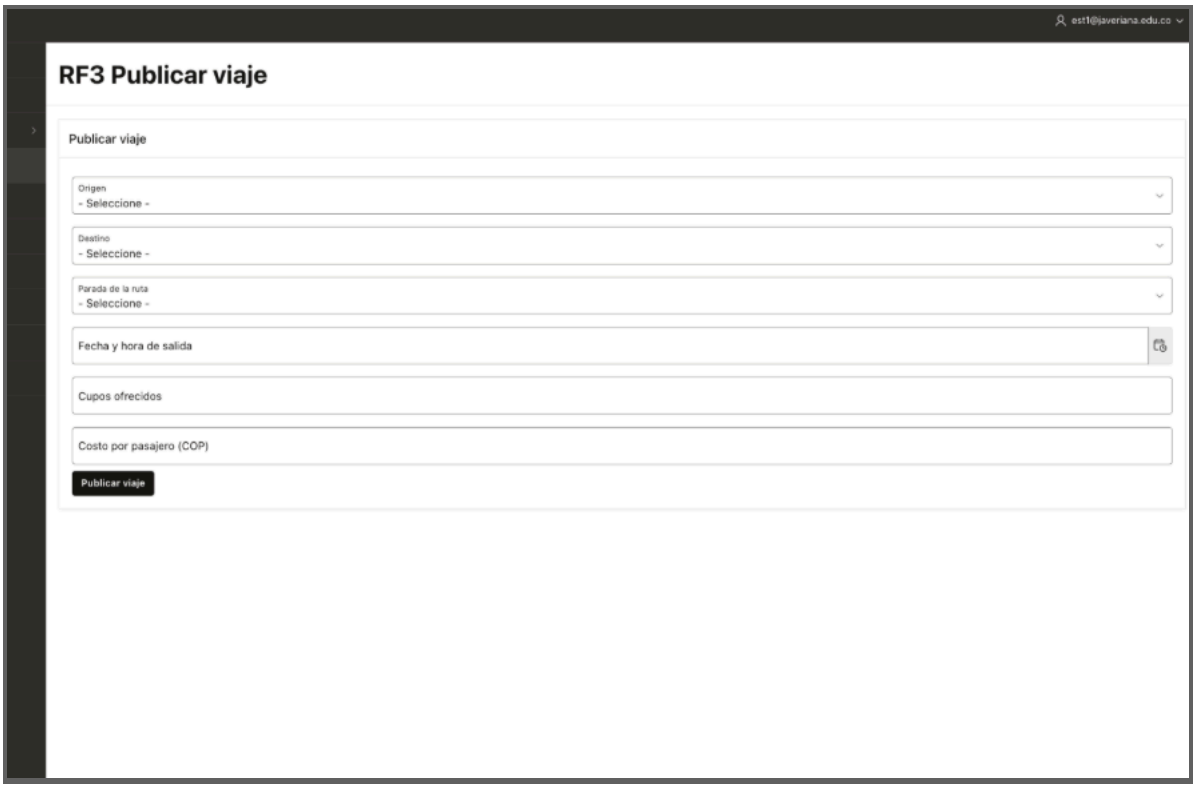
- Todas las páginas filtran por G_STUDENT_ID, nunca por un valor escrito por el usuario. Por eso un conductor solo ve y modifica sus propios viajes.
- La autorización se aplica en dos niveles: en la página (bloquea el acceso directo por URL) y en la entrada del menú (la oculta). Si solo se ocultara la entrada del menú, el usuario podría llegar escribiendo la URL.

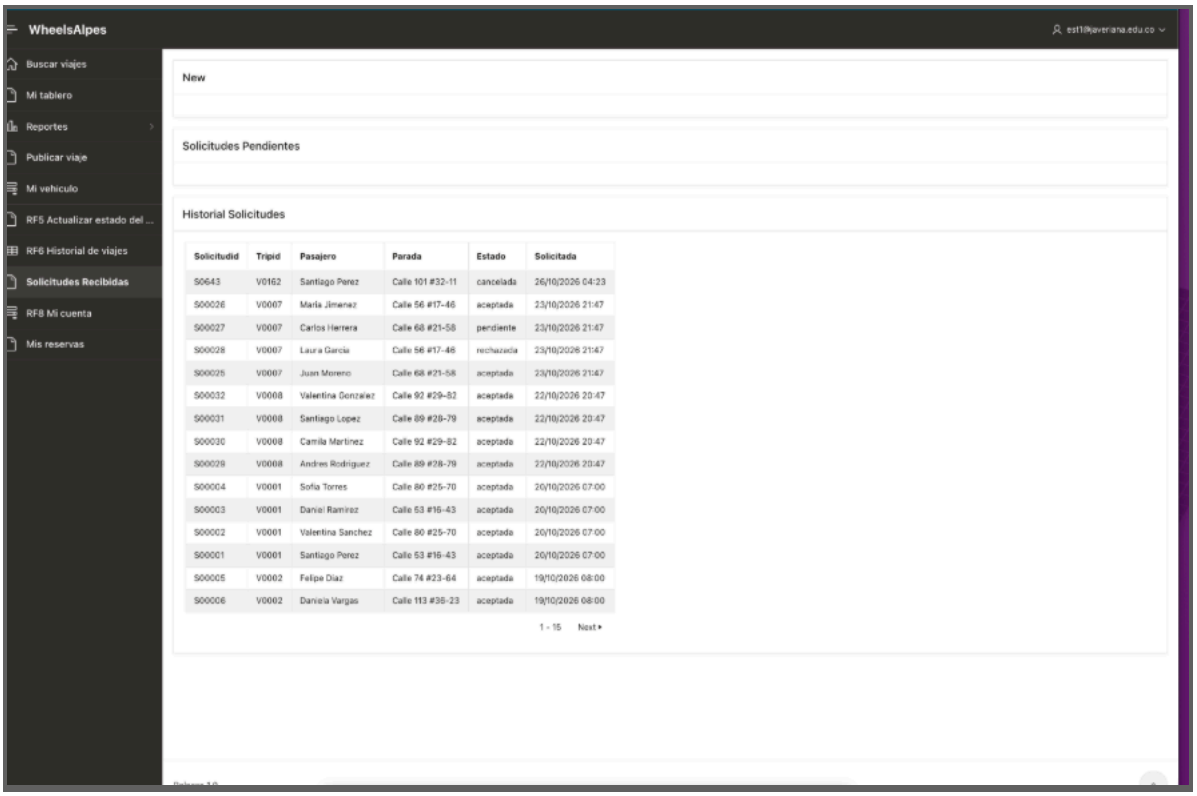


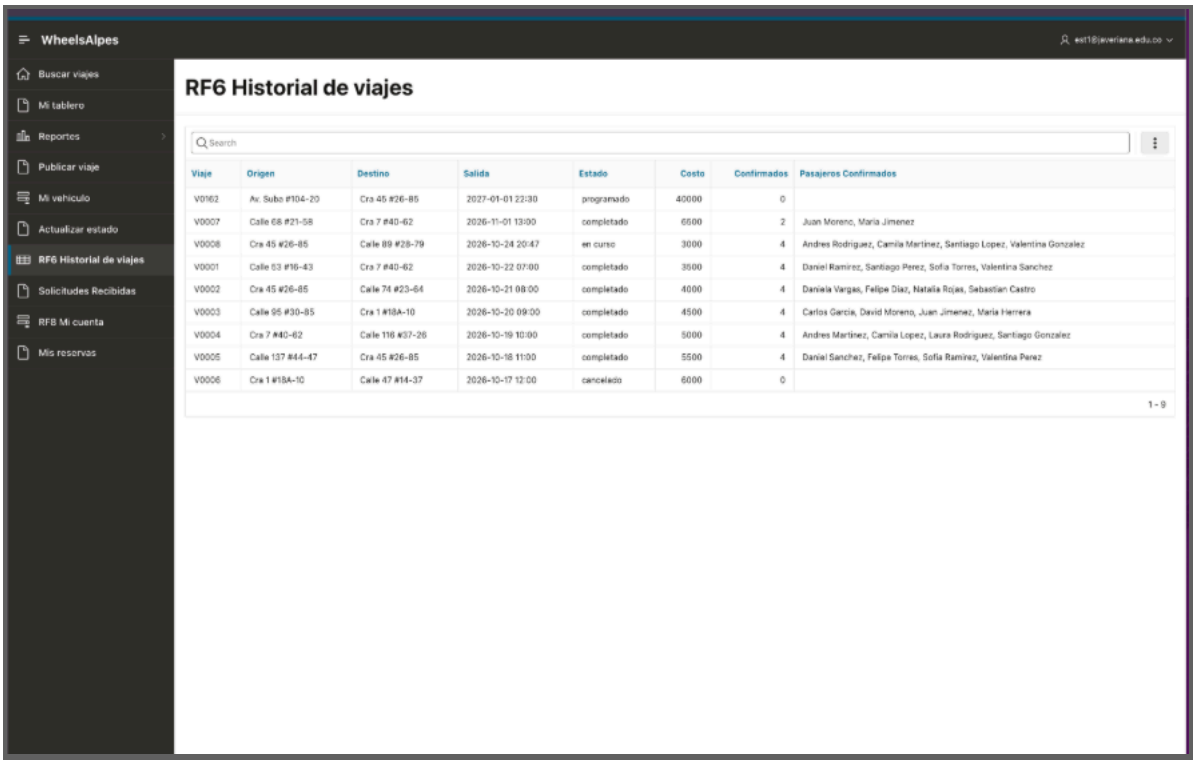


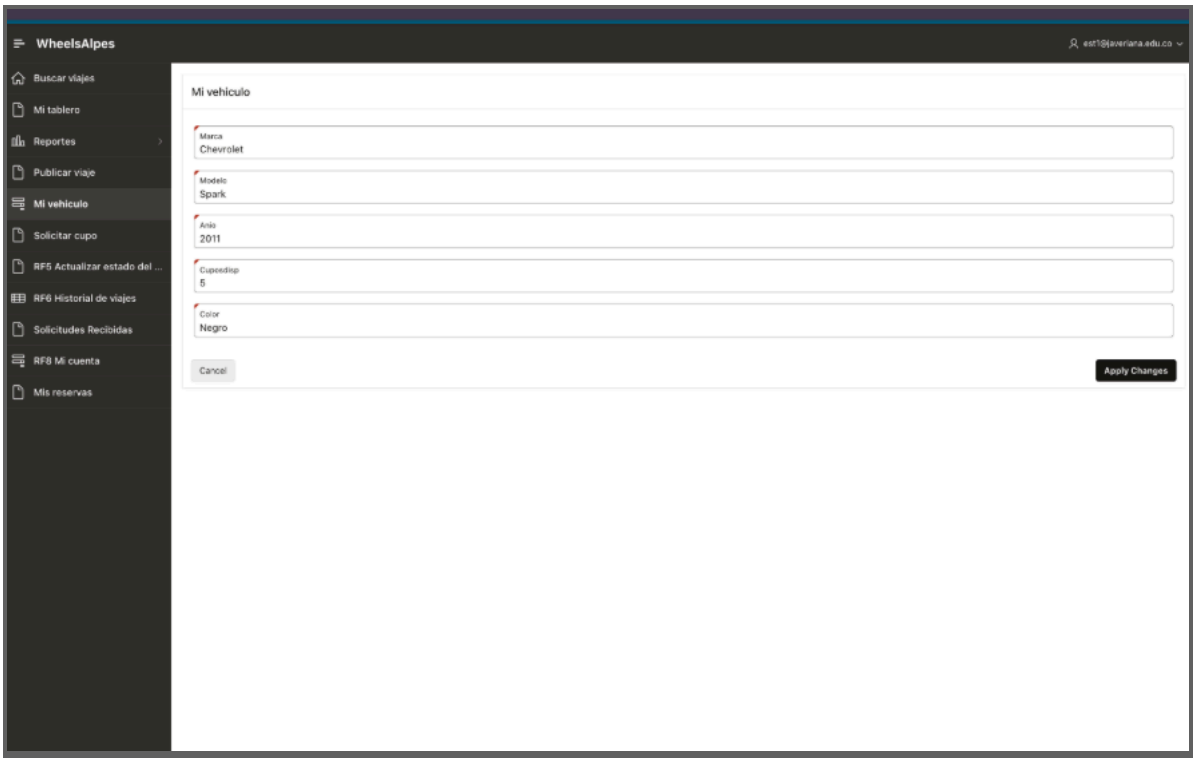
Pág.	Requerimiento	Tipo de componente
1	RF9 Buscar viajes	Región de búsqueda
2	RFA1 Tablero conductor	Tarjetas
3	RF7 Registro pasajero	Formulario

4	RF1 Registro conductor	Formulario
5, 6, 7	RFC1, RFC2, RFC3	Gráficas de barras
8	RFA2 Tablero pasajero	Tarjetas
9	RF2 Mi vehículo [confirmar]	Formulario
10	RF3 Publicar viaje	Formulario
12	RF5 Actualizar estado	Formulario + proceso PL/SQL
13	RF6 Historial de viajes	Informe interactivo
14	RF8 Mi cuenta	Formulario DML
22, 23, 24	RF10, RF4, RF11 [confirmar]	Solicitar cupo, Solicitudes recibidas, Mis reservas




WheelsAlpes
est1@javeriana.edu.co
Buscar viajes
Mi tablero
Reportes
Publicar viaje
Mi vehiculo
RF5 Actualizar estado del ...
RF6 Historial de viajes
Solicitudes Recibidas
RF8 Mi cuenta
Mis reservas
New
Solicitudes Pendientes
Historial Solicitudes
Solicitudid Tripid Pasajero Parada Estado Solicitada
S0643 V0162 Santiago Perez Calle 101 #32-11 cancelada 26/10/2026 04:23
S00026 V0007 Maria Jimenez Calle 56 #17-46 aceptada 23/10/2026 21:47
S00027 V0007 Carlos Herrera Calle 68 #21-58 pendiente 23/10/2026 21:47
S00028 V0007 Laura Garcia Calle 56 #17-46 rechazada 23/10/2026 21:47
S00025 V0007 Juan Moreno Calle 68 #21-58 aceptada 23/10/2026 21:47
S00032 V0008 Valentina Gonzalez Calle 92 #29-82 aceptada 22/10/2026 20:47
S00031 V0008 Santiago Lopez Calle 89 #28-79 aceptada 22/10/2026 20:47
S00030 V0008 Camila Martinez Calle 92 #29-82 aceptada 22/10/2026 20:47
S00029 V0008 Andres Rodriguez Calle 89 #28-79 aceptada 22/10/2026 20:47
S00004 V0001 Sofia Torres Calle 80 #25-70 aceptada 20/10/2026 07:00
S00003 V0001 Daniel Ramirez Calle 53 #16-43 aceptada 20/10/2026 07:00
S00002 V0001 Valentina Sanchez Calle 80 #25-70 aceptada 20/10/2026 07:00
S00001 V0001 Santiago Perez Calle 53 #16-43 aceptada 20/10/2026 07:00
S00005 V0002 Felipe Diaz Calle 74 #23-64 aceptada 19/10/2026 08:00
S00006 V0002 Daniela Vargas Calle 113 #36-23 aceptada 19/10/2026 08:00
1 - 15 Next ▸


WheelsAlpes
est1@javeriana.edu.co
Buscar viajes
Mi tablero
Reportes
Publicar viaje
Mi vehiculo
Actualizar estado
RF6 Historial de viajes
Solicitudes Recibidas
RF8 Mi cuenta
Mis reservas
RF6 Historial de viajes
Search
Viaje Origen Destino Salida Estado Costo Confirmados Pasajeros Confirmados
V0162 Av. Suba #104-20 Cra 45 #26-85 2027-01-01 22:30 programado 40000 0
V0007 Calle 68 #21-58 Cra 7 #40-62 2026-11-01 13:00 completado 6500 2 Juan Moreno, Maria Jimenez
V0008 Cra 45 #26-85 Calle 89 #28-79 2026-10-24 20:47 en curso 3000 4 Andres Rodriguez, Camila Martinez, Santiago Lopez, Valentina Gonzalez
V0001 Calle 53 #16-43 Cra 7 #40-62 2026-10-22 07:00 completado 3500 4 Daniel Ramirez, Santiago Perez, Sofia Torres, Valentina Sanchez
V0002 Cra 45 #26-85 Calle 74 #23-64 2026-10-21 08:00 completado 4000 4 Daniela Vargas, Felipe Diaz, Natalia Rojas, Sebastian Castro
V0003 Calle 95 #30-85 Cra 1 #18A-10 2026-10-20 09:00 completado 4500 4 Carlos Garcia, David Moreno, Juan Jimenez, Maria Herrera
V0004 Cra 7 #40-62 Calle 116 #37-26 2026-10-19 10:00 completado 5000 4 Andres Martinez, Camila Lopez, Laura Rodriguez, Santiago Gonzalez
V0005 Calle 137 #44-47 Cra 45 #26-85 2026-10-18 11:00 completado 5500 4 Daniel Sanchez, Felipe Torres, Sofia Ramirez, Valentina Perez
V0006 Cra 1 #18A-10 Calle 47 #14-37 2026-10-17 12:00 cancelado 6000 0
1 - 9


WheelsAlpes
est1@javeriana.edu.co
Buscar viajes
Mi tablero
Reportes
Publicar viaje
Mi vehiculo
Solicitar cupo
RF5 Actualizar estado del ...
RF6 Historial de viajes
Solicitudes Recibidas
RF8 Mi cuenta
Mis reservas
Mi vehiculo
Marca
Chevrolet
Modelo
Spark
Anio
2011
Cuposdisp
5
Color
Negro
Cancel
Apply Changes